

operator_runbook

BOB-008 — RuTracker Authentication Operator Runbook

Revision: 1 **Last modified:** 2026-06-10T05:12:40Z **Ticket:** BOB-008 (RuTracker CAPTCHA login — operator-blocked) **Scope:** QA evidence / operator procedure (constitution §11.4.83 docs/qa, §11.4.123 rock-solid-proof, §11.4.21 operator-blocked discipline)

This is a procedure document. The exact commands below are copy-pasteable. Do **not** run the live RuTracker steps against a CI/automation account — a CAPTCHA solve requires a human and the freeleech-only download rule still applies to any subsequent downloads.

1. Why this is operator-blocked (not structurally impossible)

RuTracker's login.php serves an **image CAPTCHA** (an `` element on the login page — see download-proxy/src/api/auth.py:134-137). Solving it requires a human to *read the distorted image with their eyes* and type the characters; there is no in-process oracle the merge service can call to decode it autonomously. This makes BOB-008 **Operator-blocked** per constitution §11.4.21 — a human must perform exactly one irreducible step (reading the CAPTCHA). It is **NOT** structurally-impossible under §11.4.112: the login fully succeeds once a human supplies the correct text (auth.py:254-262), and the equally-valid **cookie-login** path (§3 below) sidesteps the CAPTCHA entirely by reusing a session a human already established in a browser. Every non-CAPTCHA part of the flow (request schemas, routing, env-credential wiring, error handling, cookie parsing) is autonomously tested (§6). Only the live-CAPTCHA-solve is irreducibly operator-blocked.

2. Prerequisites

1. **Merge service running on port 7187.** The auth router is mounted at `/api/v1/auth` (download-proxy/src/api/__init__.py:239 imports auth_router; __init__.py:252 mounts it with prefix="/api/v1" on top of the router's own prefix="/auth" from auth.py:23). The app is served by download-proxy/src/main.py on MERGE_SERVICE_PORT, default 7187 (api/__init__.py:100). Confirm it is up:

```
curl -s http://localhost:7187/api/v1/auth/status | python3 -m json.tool
```

2. **RUTRACKER_USERNAME / RUTRACKER_PASSWORD set in the environment** that the merge-service process sees. Env priority (per project CLAUDE.md "Environment Variables"): shell env → `./env` → `~/qbit.env` → container env. See `.env.example:21-22` for the keys (RUTRACKER_USERNAME, RUTRACKER_PASSWORD; `.env.example:8` shows the `~/bashrc` export pattern, `.env.example:19-20` notes the RUTRACKER_USER/RUTRACKER_PASS aliases). The service loads these via `SearchOrchestrator._load_env()` which consults `/config/.env`, `~/qbit.env`, `/root/.qbit.env` as fallbacks (`search.py:632-636`).

Credentials are read server-side from env, NEVER from the request body. Both the CAPTCHA and the cookie path read `os.getenv("RUTRACKER_USERNAME")` / `os.getenv("RUTRACKER_PASSWORD")` inside the handler (`auth.py:115-116` for `/captcha`, `auth.py:221-222` for `/login`). The request body carries only the CAPTCHA text/token or the cookie string — never your password. (Mirrors the explicit NNMClub note at `auth.py:412-414`.)

3. **RUTRACKER_MIRRORS** (optional). If unset, the base URL defaults to `https://rutracker.org` (`auth.py:124`, `auth.py:315`). `.env.example:27` documents the override.

3. RECOMMENDED PATH — cookie login (sidesteps the CAPTCHA)

This is the preferred procedure. A human logs in once in a normal browser (solving the CAPTCHA there), then hands the resulting session cookie to the service. No CAPTCHA decoding in `curl` is involved.

3a. Obtain `bb_session` from a browser

1. Open `https://rutracker.org/forum/login.php` in a desktop browser and log in normally (you will solve the CAPTCHA in the browser UI).
2. Open DevTools → **Application/Storage** → **Cookies** → `https://rutracker.org`.
3. Copy the value of the cookie named `bb_session`. (This is the only cookie the endpoint requires — `auth.py:309`.)

3b. POST it to the cookie-login endpoint

```
curl -s -X POST http://localhost:7187/api/v1/auth/rutracker/
      cookie-login \
      -H 'Content-Type: application/json' \
      -d '{"cookie_string":"bb_session=PASTE_YOUR_VALUE_HERE"}'
```

The body field is `cookie_string` — the only field of `CookieLoginRequest` (`auth.py:52-53`). You may paste the full Cookie: header string (e.g. `"bb_session=...; bb_data=...; cf_clearance=..."`); the handler splits on `;` and `=` and keeps every pair (`auth.py:303-307`), but it **must contain** `bb_session` or it is rejected.

3c. Responses (cited)

HTTP	When	Source
200 {"authenticated": true, "message": "Successfully authenticated with RuTracker via cookie."}	Cookie verified live (<code>/forum/index.php</code> returns the page containing <code>id="logged-in-username"</code>) and stored	<code>auth.py:329</code> , <code>auth.py:344-347</code>
400 Cookie string must contain <code>bb_session</code> cookie.	No <code>bb_session</code> in the parsed cookie jar	<code>auth.py:309-313</code>
401 Cookie is invalid or expired.	Cookie present but the verification fetch does NOT find <code>id="logged-in-username"</code> (i.e. not actually logged in)	<code>auth.py:329-333</code>
502 Could not verify cookie: <code><err></code>	The verification HTTP request to RuTracker itself	<code>auth.py:336-337</code>

HTTP	When	Source
	failed (network/ timeout)	

On 200, the session cookie is stored in the orchestrator's encrypted in-memory store under key "rutracker" (auth.py:339-342).

4. CAPTCHA PATH (fallback — currently awkward, see the friction note)

Use this only if you cannot obtain a browser cookie. The flow is: GET /captcha → human reads the image → POST /login.

4a. Fetch the CAPTCHA

```
curl -s http://localhost:7187/api/v1/auth/rutracker/captcha > /
    tmp/bob008_captcha.json
python3 -m json.tool /tmp/bob008_captcha.json | head -8
```

A CAPTCHA-required response looks like (auth.py:200-206):

```
{
  "captcha_required": true,
  "authenticated": false,
  "captcha_image": "...",
  "captcha_token": "<urlsafe-token>",
  "message": "CAPTCHA detected. Submit the text via /auth/
    rutracker/login."
}
```

Other possible outcomes from /captcha: - {"captcha_required": false, "authenticated": true, ...} — RuTracker let the service log in without a CAPTCHA; you're done (auth.py:157-161). Verify with §5. - {"captcha_required": false, "authenticated": false, ...} — no CAPTCHA found; try logging in directly (auth.py:172-177). - 400 — RUTRACKER_USERNAME and RUTRACKER_PASSWORD not configured (auth.py:118-122). - 502 — could not parse the CAPTCHA form fields or could not fetch the login page (auth.py:182-186, auth.py:209-211).

4b. Decode the base64 image to a PNG a human can view

```
python3 - <<'PY'
import base64, json
data = json.load(open("/tmp/bob008_captcha.json"))
```

```

url = data["captcha_image"] # "data:image/
    png;base64,...."
b64 = url.split(",", 1)[1] # strip the
    data-URL prefix
open("/tmp/bob008_captcha.png", "wb").write(base64.b64decode(b64))
print("token:", data["captcha_token"])
print("wrote /tmp/bob008_captcha.png – open it and read the text")
PY
open /tmp/bob008_captcha.png # macOS; use xdg-open on Linux

```

4c. **FRICTION — /login needs cap_sid + cap_code_field that /captcha does NOT return**

POST /rutracker/login validates against CaptchaLoginRequest, whose **required** fields are cap_sid, cap_code_field, captcha_text, captcha_token (auth.py:45-49). **But the /captcha response above only returns captcha_image + captcha_token** — it does NOT return cap_sid or cap_code_field. Those two values are parsed server-side from the login page (auth.py:179-180) and stored **server-side** inside the pending-CAPTCHA store, keyed by the captcha_token (auth.py:193-198); the actual login handler pops them back out of that store by token (auth.py:230-237) and ignores... no — it uses the cap_sid and cap_code_field **from the request body** (auth.py:247-248), even though it already holds the authoritative copies under the token.

Net effect for a curl operator: the values the body requires are not handed back to you, so you cannot correctly populate cap_sid / cap_code_field from the curl response alone. To complete this path by hand you would have to scrape name="cap_sid" value="..." and name="cap_code_..." out of https://rutracker.org/forum/login.php yourself (the same regexes the server uses: auth.py:179-180). This is why **the cookie path (§3) is preferred** — it has no such hidden-field requirement. A future BOB-008 follow-up should either echo cap_sid/cap_code_field in the /captcha response, or have /login use the token-stored copies and require only captcha_text + captcha_token from the body.

4d. **Submit (once you have all four fields)**

```

curl -s -X POST http://localhost:7187/api/v1/auth/rutracker/login
\
-H 'Content-Type: application/json' \
-d '{
    "cap_sid": "<scraped from login page>",
    "cap_code_field": "<e.g. cap_code_xxxxx, scraped>",
    "captcha_text": "<what you read in the PNG>",
    "captcha_token": "<from step 4a>"
}'

```

Responses: - 200 {"authenticated": true, "message": "Successfully authenticated with RuTracker."} — success, session stored (auth.py:254-262). - 200 {"authenticated": false, "captcha_required": true, "captcha_image": ..., "captcha_token": <new>, "message": "Wrong CAPTCHA. A new one has been generated."} — wrong text; a fresh CAPTCHA + token are returned, retry from 4b (auth.py:279-285). - 200 {"authenticated": false, "message": "Login failed. Check credentials."} — bad credentials (auth.py:287-290). - 400 Invalid or expired captcha_token... — the token wasn't in the pending store (expired after PENDING_CAPTCHAS_TTL_SECONDS, default 900s, auth.py:31; or already consumed — it is popped on use, auth.py:230). Fetch a new one (auth.py:230-235). - 400 RUTRACKER_USERNAME and RUTRACKER_PASSWORD not configured (auth.py:224-228). - 500 Login failed: <err> — unexpected error during the login request (auth.py:293-295).

5. Post-login verification

Confirm the session is live:

```
curl -s http://localhost:7187/api/v1/auth/rutracker/status |  
python3 -m json.tool
```

Expect {"authenticated": true, "status": "active", "message": "RuTracker session is active."} — this is returned only when the stored bb_session cookie actually loads /forum/index.php and the page contains id="logged-in-username" (auth.py:89-94). Other states: no_session (nothing stored, auth.py:61-66), no_cookie (session present but no bb_session, auth.py:69-74), expired (cookie no longer logs in, auth.py:95-99), error (verification request failed, auth.py:100-105).

The session does NOT survive a merge-service restart. Sessions are stored in SearchOrchestrator._tracker_sessions, an EncryptedSessionStore wrapping an in-memory TTLCache (search.py:586-588). Per its own docstring the encryption key is per-process ephemeral and “sessions live only in the in-memory TTLCache and never outlive the process” (search.py:508-511); the cache also has a TTL (search.py:586-587, default from _max_searches/_ttl). Therefore **BOB-008 must be re-run after any merge-service restart** (and after the TTL expires). Pinning SESSION_ENCRYPTION_KEY (search.py:519) only preserves the key, not the in-memory entries — a restart still loses every stored session.

6. Autonomous coverage (what is already tested without a human)

Everything except the live-CAPTCHA-solve is covered by existing automated tests, so only the irreducible human step is operator-blocked:

Area	Test(s)	Notes
Request schemas (CaptchaLoginRequest / CookieLoginRequest field names, required-field validation)	tests/unit/test_auth_models.py:18-37, tests/unit/test_auth.py:29-46	Asserts the exact cap_sid/ cap_code_field/ captcha_text/ captcha_token and cookie_string fields
Router exists / mounted; pending-captcha store is a bounded mapping	tests/unit/test_auth.py:13-27	
/rutracker/status all states (no_session / no_cookie / active / expired / error)	tests/unit/ test_auth_coverage.py:36-120	Drives the status handler with mocked aiohttp
/captcha flow: missing-creds 400, CAPTCHA detected + token persisted, no-CAPTCHA direct login	tests/unit/api_layer/ test_auth_coverage_extra.py:55-95	Asserts token stored with correct cap_sid
Cookie login: missing bb_session 400, invalid cookie 401, valid cookie 200	tests/unit/ test_auth_coverage.py:149-208	Cookie parsing + all three response paths
/auth/status aggregate (all trackers)	tests/unit/ test_auth_coverage.py:208-...	
CAPTCHA cache bounded + TTL-evicting (memory-safety)	tests/security/ test_captcha_cache_bounded.py:21-50	Confirms TTLCache, maxsize ≤ 1024, ttl 900
Credentials read from env (not body); tracker enabled/ authenticated only when both creds set	tests/unit/ test_credential_env_wiring.py:115-174	Proves server-side env wiring

Area	Test(s)	Notes
Auth-bypass / brute-force / no-credentials hardening	tests/security/test_auth_bypass.py:27-101	

Irreducibly operator-blocked: only the human act of reading the distorted CAPTCHA image and typing the characters (auth.py:134-137, consumed at auth.py:247-248). Every surrounding mechanism is autonomously verified above.

Sources verified

Files read in this session (constitution §11.4.99):

- download-proxy/src/api/auth.py:1-532 — all RuTracker auth handlers, request models, response bodies, status codes (lines cited inline throughout).
- download-proxy/src/api/__init__.py:86-100, 239-253 — FastAPI app, MERGE_SERVICE_PORT default 7187, auth_router import + mount at /api/v1 + /auth.
- download-proxy/src/main.py:10-55 — uvicorn entry point serving the FastAPI app.
- download-proxy/src/merge_service/search.py:496-563 (EncryptedSessionStore, ephemeral key, in-memory TTLCache), :566-593 (orchestrator session-store wiring), :622-648 (_load_env fallback paths).
- .env.example:8, 19-22, 27 — RUTRACKER_USERNAME/RUTRACKER_PASSWORD/RUTRACKER_MIRRORS keys and aliases.
- tests/unit/test_auth.py, tests/unit/test_auth_models.py, tests/unit/test_auth_coverage.py, tests/unit/api_layer/test_auth_coverage_extra.py, tests/security/test_captcha_cache_bounded.py, tests/security/test_auth_bypass.py, tests/unit/test_credential_env_wiring.py — autonomous coverage cited in §6 (line ranges inline).